INT161 · Basic Backend Development

Template and CRUD Practice

บันทึกโปรเจกต์เป็น template, copy ทั้งสาม layer ไปทำ resource ใหม่, ทดสอบทีละ layer และอ่าน error ครั้งแรก

Project Template | A New Resource in Three Layers | Test One Layer at a Time

Reading Your First Errors | Lab: Actor CRUD | อภิธานศัพท์

01

Project Template

อาจารย์ตั้งใจสอนเรื่อง **exception** วันนี้ แต่ใช้เวลาทั้งคลาสกับเรื่องนี้: **project template** แล้วสร้าง **resource** ใหม่จากมัน error โผล่ระหว่างทาง ส่วนเนื้อหาเต็มย้ายไปคลาสถัดไป

- template มีโฟลเดอร์ครบ: configs (การตั้งค่า database), repositories, **services** และ **routes** ทุกอันมีไฟล์ตัวอย่าง รวมถึง app.js
- example **repository** เก็บ SQL ไว้ใน constant หนึ่งอันต่อหนึ่งการทำงาน และใช้ตัวแปรสำหรับชื่อ table เวลาใช้ซ้ำแค่เปลี่ยนชื่อ table กับ **entity** นั่นคือแนวคิด
- อย่าแก้ตัวโปรเจกต์ template เอง ให้บันทึกไว้ แล้วทำงานในโปรเจกต์ที่สร้างจากมัน

1. แตก zip ของ template และเช็คว่าโฟลเดอร์ไม่ซ้อนกันสองชั้น
2. วางโฟลเดอร์ในที่ที่ IDE เก็บโปรเจกต์ แล้วเปิดขึ้นมา
3. รัน **npm** install ในโปรเจกต์ โฟลเดอร์ node_modules สร้างใหม่ได้ จึงไม่ควร copy ไปมา
4. WebStorm: Tools, Save Project as Template ตั้งชื่อ
5. หลังจากนั้น New Project จะมี template ของคุณให้เลือก เลือก ตั้งชื่อโปรเจกต์ แล้วโฟลเดอร์และ code จะถูก copy ให้

after creating a project from the template

```
$ npm install
# then set the database name in configs/db-config.js
```

> **ใช้ AI**
>
> ไม่แน่ใจว่า VS Code บันทึก template อย่างไร ถาม AI ได้ ใช้สร้างความรู้ ไม่ใช่ให้ทำงานแทน บริษัทไม่ได้จ่ายค่า token ให้ทุกคนเสมอ จึงต้องทำงานได้เองโดยไม่พึ่งมัน

02

A New Resource in Three Layers

database ที่ใช้สาธิตคือ Sakila และ resource ใหม่คือ country สูตรเดียวกันใช้ได้กับทุก table

1. Repository

copy example repository เปลี่ยนชื่อเป็น country-repository แล้วเปลี่ยนชื่อ table และ entity

2. Service

copy example service เปลี่ยนชื่อ และให้ชี้ไปที่ country repository

3. Route

copy example route เปลี่ยนชื่อ และให้ชี้ไปที่ country service

4. App

ลงทะเบียน **router** ใต้ path ของมันใน app.js

repositories/country-repository.js

```
import db from '../configs/db-config.js';
const TABLE_NAME = 'country';
const ENTITY_NAME = 'Country';
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`;
const FIND_ONE_SQL = `SELECT * FROM ${TABLE_NAME} WHERE country_id = ?`;
const CREATE_SQL = `INSERT INTO ${TABLE_NAME} SET ?`;
const UPDATE_SQL = `UPDATE ${TABLE_NAME} SET ? WHERE country_id = ?`;
const REMOVE_SQL = `DELETE FROM ${TABLE_NAME} WHERE country_id = ?`;
export async function findAll() {
  const [rows] = await db.query(FIND_ALL_SQL);
  return rows;
}
export async function findOne(id) {
  const [rows] = await db.query(FIND_ONE_SQL, [id]);
  if (rows.length === 0) throw new Error(`${ENTITY_NAME} with id ${id} was not found`);
  return rows[0];
}
```

- SET ? รับ object ทั้งก้อน ชื่อ field ของ object ต้องตรงกับชื่อคอลัมน์ mysql2 สร้างรายการ SET ให้
- table มี key แบบ **auto-increment** และ last_update ที่ใส่เวลาให้เอง ส่งแค่ชื่อประเทศ
- หลัง insert repository อ่านแถวใหม่กลับมาด้วย insert id จึงตอบเป็นแถวเต็ม ไม่ใช่คำตอบดิบของ database
- เปลี่ยนชื่อให้ระวัง ถ้าชื่อซ้ำหลายที่ ใช้ find and replace (**refactor**) แทนการแก้ทีละตัว ชื่ออย่าง id ใน template อาจไม่ตรงกับ table ของคุณ

app.js

```
import express from 'express';
import countryRouter from './routes/country-route.js';
const app = express();
app.use(express.json()); // before the router, or req.body is undefined
app.use('/countries', countryRouter);
app.listen(3000);
```

03

Test One Layer at a Time

ทดสอบทีละ layer ก่อนต่อ layer ถัดไป ถ้าเขียนทั้งสามแล้วทดสอบครั้งเดียว จะบอกไม่ได้ว่า layer ไหนพัง

1. Repository

import เข้าไฟล์ทดสอบเล็ก ๆ แล้วเรียกแต่ละฟังก์ชัน พิมพ์ผล

2. Service

ไฟล์ทดสอบเดิม แต่เรียก service ผลควรเหมือนกัน

3. Route

เรียกจาก script ไม่ได้ เพราะต้องใช้ **HTTP request** จริง รันแอปแล้วใช้ **Postman**

tests/test-country-repository.js

```
import * as repo from '../repositories/country-repository.js';
console.log(await repo.findAll());
console.log(await repo.findOne(94));
// console.log(await repo.create({ country: 'Thailand' })); // no id: the database makes it
```

- update และ delete: ทดสอบกับ id ที่สร้างเอง อย่าแตะแถวที่คนอื่นอาจใช้
- เช็คผลของการเปลี่ยนแปลงใน database ด้วย ไม่ใช่ดูแค่คำตอบที่ code พิมพ์

route layer คือ **controller** อ่าน request และตอบด้วย **status code** มาตรฐาน

GET /countries

200 with an array

`GET /countries/:id`

200 with one object

`POST /countries`

201 with the new row

`PUT /countries/:id`

200 with the changed row

`DELETE /countries/:id`

204 with no body

- หลัง **express**.json() **body** เป็น object แล้ว ส่ง req.body ต่อได้เลยถ้าเชื่อใจ เรายังไม่ validate จึงสมมติว่า **front end** ส่งมาถูก
- delete ตอบ 204 และไม่ส่งอะไร ถ้าอยากพิสูจน์ว่าลบได้ ให้ดึง id เดิมอีกครั้งและคาดว่าจะไม่เจอ

04

Reading Your First Errors

error จริงครั้งแรกโผล่ในคลาส template ใช้ id แต่ table เรียก key ว่า country_id กฎของอาจารย์: อ่าน error message เพราะเป็นเบาะแสเดียวที่เชื่อถือได้ อย่าเทียบ code กับต้นฉบับแล้วเดา

Message | บรรทัดแรกที่เป็น code ของเรา | Exit code

```
Error: Unknown column 'id' in 'where clause'
    at async findOne (src/repositories/country-repository.js:24:29)
Process exited with code 1
```

บอกว่า

บอกว่าผิดอะไร database ไม่รู้จักคอลัมน์ชื่อ id

ดังนั้นเรา

เปิดตารางดูชื่อคอลัมน์จริง ในที่นี้ key ชื่อ country_id จึงต้องเปลี่ยน id ใน template

เป็นตัวอย่าง ชื่อไฟล์และบรรทัดของคุณจะต่างไป

- ใน IDE ไฟล์ใน **stack trace** เป็นลิงก์ คลิกเพื่อไปที่บรรทัดนั้น
- ไม่เจอข้อมูลก็เป็น error การลบ id 110 สองครั้ง ครั้งที่สองไม่เจออะไร findOne จึง **throw** ถ้าไม่จัดการ **Node** จะจบด้วย code 1 และ **server** ตาย ใน production แปลว่าแอปล่ม

> ⚠ **คลาสถัดไป**
> การจัดการ error เหล่านี้ให้ server ไม่ตายและตอบด้วย status code ที่เหมาะสม เป็นหัวข้อของคลาสถัดไป

05

Lab: Actor CRUD

งานในคลาส: ทำ **CRUD** ของ resource actor ใน Sakila มีสี่คอลัมน์ ต้องจัดการสองอัน คือ first_name กับ last_name ส่วน id เป็นอัตโนมัติ และ last_update ใส่เองให้

- สาม layer: repository, service, route ห้า **endpoint**: get all, get by id, create, update, remove
- ส่ง zip ของโปรเจกต์และ PDF ของภาพหน้าจอ หนึ่งภาพต่อ endpoint ให้เห็น **URL** และข้อมูล get all ภาพเดียวพอ เพิ่มคำอธิบายสั้น ๆ ว่าทำอะไร
- ลบ node_modules ก่อน zip มันใหญ่มากและผู้ตรวจต้องติดตั้งใหม่อยู่แล้ว ถ้าลืมจะถูกหักคะแนน
- พิสูจน์การลบ: แสดง 204 แล้วดึง id เดิมและแสดงว่าไม่มีแล้ว
- คุยและถามเพื่อนได้เหมือนเพื่อนร่วมงาน แต่ห้าม copy code
- เสร็จเร็ว ลอง resource city ยากกว่าเพราะมี relationship

> 📋 **ที่อาจารย์พูดถึงด้วย**
> โปรเจกต์ที่ใช้ Express รันด้วย Bun ได้ด้วย อาจารย์บอกว่าเร็วกว่า ลองดูได้

06

อภิธานศัพท์

auto-increment
: database นับ id เพิ่มให้เอง ไม่ต้องส่ง id ตอน INSERT และไม่ซ้ำ

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

controller
: layer ที่รับ HTTP request และส่ง response แล้วส่งต่องานจริงให้ layer อื่น

CRUD
: Create, Read, Update, Delete: สี่อย่างพื้นฐานที่ทำกับข้อมูล

endpoint
: method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

entity
: ข้อมูลใน database ตามที่เก็บ: แถวของ table พร้อมทุกคอลัมน์

exception
: เหตุการณ์ที่ขัดขวางลำดับการทำงานปกติของโปรแกรม ถูกโยนออกมาเป็น object ที่เก็บรายละเอียด

express
: web framework ขนาดเล็กของ Node.js เพิ่ม routing และ middleware บน http module ที่มีมาให้

front end
: ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

HTTP
: HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

Node
: runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

npm
: Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

Postman
: เครื่องมือส่ง HTTP method ใดก็ได้ไปที่ API ใช้ทดสอบก่อนที่ front end จะเสร็จ

project template
: โปรเจกต์ตั้งต้นที่บันทึกไว้ โปรเจกต์ใหม่ที่สร้างจากมันจะมีโฟลเดอร์และ code พื้นฐานให้แล้ว

refactor
: ปรับโครงสร้างหรือชื่อใน code โดยไม่เปลี่ยนสิ่งที่มันทำ เช่น เปลี่ยนชื่อ field ทุกที่

repository
: layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

request
: สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

resource

สิ่งที่ API ให้เข้าถึง เช่น product หรือ customer ในวิชานี้ตรงกับ table ใน database

router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

routes

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

services

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

stack trace

รายการการเรียกฟังก์ชันพร้อมไฟล์และบรรทัดที่นำไปสู่ error บรรทัดบนสุดคือที่ที่ต้องดูก่อน

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error

throw

โยน error ออกมา โยนได้เฉพาะ object ของ Error

URL

ที่อยู่ของ resource ประกอบด้วย protocol, host, port และ path เช่น http://localhost:3000/user